



Tuka Curious Minds Electrical Configuration

Robot Name: TURBO Ceros-T

Category: Robo Wars

Weight Classification: 5.kg Max Limit

Footprint Matrix: 420mm (Width) x 420mm (Length) x 55mm (Sleek Casing Height)

Electrical Configuration (Control Hub Wiring)

- **Port 0:** leftDrive Motor
- **Port 1:** rightDrive Motor
- **Port 2:** bladeMotor
- **Port 3:** (Reserved for future upgrades, like a launcher or lift)
- **I2C Port:** imu (The built-in Rev Hub IMU)

Software Configuration Methodology (REV Hardware Client)

This is where your students often get stuck!

- Open the REV Hardware Client on your laptop and connect to the Control Hub.
- **Name Motors:** Exactly leftDrive, rightDrive, and bladeMotor. Select "Core Hex" as the motor type so the encoders read correctly.
- **Name IMU:** imu.
- **IMU Orientation (CRITICAL):** You must select the LogoFacingDirection and UsbFacingDirection in the configuration so the IMU knows which way is "Up" and "Forward" relative to the robot's physical frame. If this is wrong, the robot will violently spin in circles when you try to drive straight.

Code Functionality Explained (For the Team)

Here is exactly what the Java code is doing, broken down by section so the students can understand the logic:

Arcade Drive

```
double drive = -gamepad1.left_stick_y;
```

```
double turn = gamepad1.right_stick_x;
```

- **Functionality:** The Left stick moves forward and backward (inverted because pushing the stick up yields a negative value). The Right stick turns the robot.
- **The Math:** $\text{leftPower} = \text{drive} + \text{turn}$ and $\text{rightPower} = \text{drive} - \text{turn}$. This allows the robot to pivot in place when the turn is high and the drive is 0. The 0.8 multiplier caps the speed so the robot doesn't flip over or hit walls too fast.

The IMU "Anti-Push" System (The Secret Weapon)

```
double currentYaw =
```

```
imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.DEGREES);
```

```
double headingError = currentYaw; // Assuming target is 0 degrees
```

```
double correction = headingError * HEADING_P;
```

```
turn = turn + correction;
```

- **Functionality:** This is pure, simplified Proportional Control (P-Control). The IMU constantly asks: *"Am I facing 0 degrees?"* If an opponent pushes the robot sideways, the robot rotates slightly (e.g., to 10 degrees). The code sees this error (10 degrees) and calculates a correction ($10 * 0.05 = 0.5$ power).
- **How it "fights back":** It adds this correction to the turn input, automatically spinning the robot back to its original heading (0 degrees). This makes the robot act like it is "stuck to the floor" and very difficult to push around.

The Boolean Lock for the Blades

```
boolean bladeRunning = false;
```

```
if (gamepad1.y) bladeRunning = true;
```

```
if (gamepad1.x) bladeRunning = false;
```

```
if (bladeRunning) bladeMotor.setVelocity(BLADE_RPM, AngleUnit.RPM);
```

```
else bladeMotor.setPower(0.0);
```

- **Functionality:** This is a "toggle switch." You press Triangle once, and bladeRunning becomes true. Even if you let go, it stays true. It doesn't stop until you press the Square/Cube button, which sets bladeRunning to false.

- **RPM Control:** Using `setVelocity(6000, RPM)` tells the motor controller to constantly fight to maintain exactly 6000 RPM, even if the blades are hitting something. This gives the cutters incredible bite.

The Zero Power Brake (The Safety Feature)

```
leftDrive.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.BRAKE);
```

```
bladeMotor.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.BRAKE);
```

- **Functionality:** When the joystick is released (no power), a normal motor "floats" and rolls. **BRAKE** actively shorts the motor terminals to create an electromagnetic lock. The robot stops *dead* in its tracks. This prevents the robot from drifting into opponents and ensures the blades stop instantly when the toggle is turned off (making it safe for human handling).

Telemetry

```
telemetry.addData("Left Speed (ticks/sec)", leftDrive.getVelocity());
```

- **Functionality:** This prints the live encoder speed and Yaw degrees to the driver's phone. You use this during testing to see if the drivetrain is struggling (Low ticks/sec) or if the IMU is fighting hard to hold its heading.